<u>CAD for VLSI Design</u>

指導教授: 陳聿廣

PA2

Partitioning

(結報)

系級 電機四 B

學號 110501544

姓名 翁瑄妤

1. Compilation and Execution

主要在工作站上執行程式，其系統為 Linux 3.10.0、Host 為 140.115.71.87。

執行指令分為以下步驟:

(1) 打開所屬資料夾 (所需資料皆在 PA2 的資料夾裡)

cd PA2/

(2) 執行 Makfile 檔案：

```makefile
# PA2 cpp compiler
.PHONY: all run clean
all: 110501544_PA2.cpp
	g++ -std=c++11 110501544_PA2.cpp -o exe
run:
	./exe $(input) $(output)
clean:
	rm *.o
```

- 將作業 cpp 檔輸出成執行檔.o 檔

  make all
- 執行輸出檔，生成輸出檔案(以 case1 為例)

  make run input=case1 output=case1.out
- 清除執行檔

  make clean

(3) 執行 CHECKER 檔確認輸出檔案內容是否正確 (以 case1 為例)

chmod 700 PA2_CHECKER

./PA2_CHECKER case1 case1.out

2. Completion

已完成以下要求:

- 解析符合格式的 netlist 輸入檔
- 完整實作 FM Algorithm
- 維持切割平衡條件：|size(A) – size(B)| <= n/5
- 完成輸出格式：cut_size + A/B cell ID 列表
- 能於 10 分鐘內輸出檔案

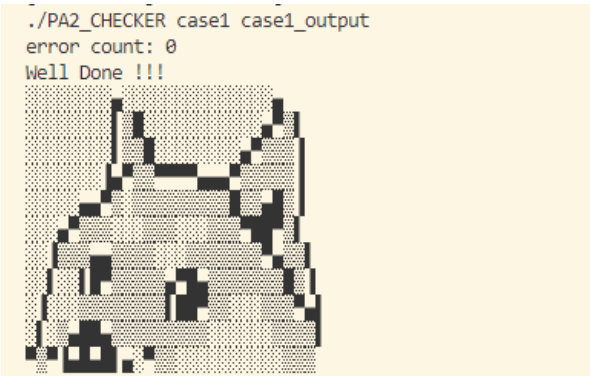
以下為執行結果：

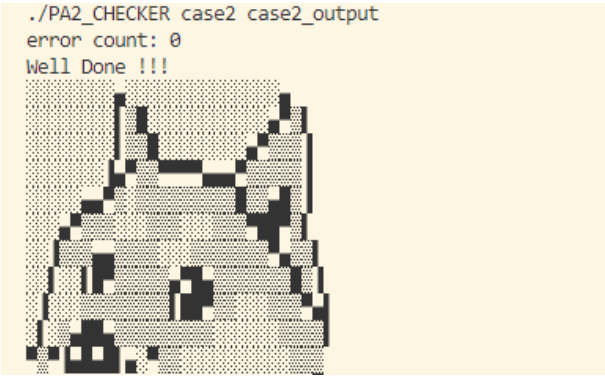
- case1

```
cut_size 3
A
c4
c11
c5
c9
c6
c2
B
c8
c10
c7
c1
```

- case2

```
cut_size 881
A
c131
c116
c101
c90
c76
c57
c199
c149
c35
c155
c26
c123
```

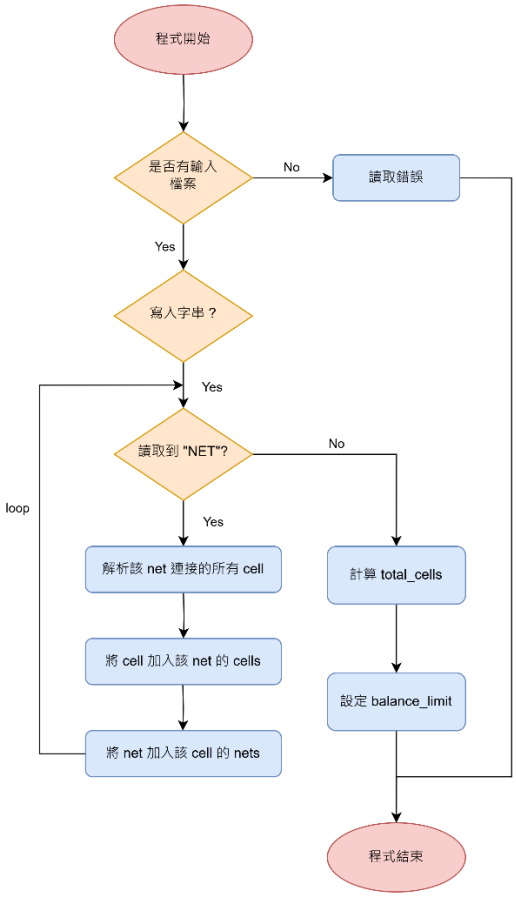




由上面兩張照片可得知，執行出來的兩份輸出檔案皆為正確，且指令皆以比照投影片要求執行，因此這份檔案已完成作業中的所有要求。

3. Explanation

(1) Input_Parse()：從輸入檔案中解析 netlist，建立 cell 與 net 的結構，並初始化區域資訊與平衡限制條件。



流程:

- 逐行讀入檔案內容。

```
ifstream inputfile(filename, ios::in);
if(!inputfile)
{
    cerr << "File could not be opened" << endl;
    exit(0);
}
```

- 每當遇到 NET，讀取其後的 net 名稱與其包含的 cell。
- 更新每個 cell 的 net 列表與每個 net 的 cell 列表。
- 將未分配 region 的 cell 預設設定為左邊 (region = 0)。

```
while(inputfile >> line){
    if(line == "NET"){
        inputfile >> line;
        string net_name = line;
        getline(inputfile, line);
        char *token = strtok(&line[0], "c{ }");
        // Save the information of each cell
        while (token != NULL){
            int id = stoi(token);
            if(cells[id].region == -1){
                cells[id].region = 0; // default region
                cell_count++;
            }
            cells[id].nets.insert(net_name);
            nets[net_name].cells.insert(id);
            token = strtok(NULL, "c{ }");
        }
    }
}
```

- 計算平衡限制（balance_limit = total_cells / 5）。

```
balance_limit = cell_count / 5;     // constraint: |A - B| <= total_cells/5
if(balance_limit < 1) balance_limit = 1; // at least 1
```

(2) Initial Solution (在這程式中主要用方法二進行) -- **BONUS**

a. 方法一: Initial_Partition_Random(): 隨機將所有 cell 分配到左右兩個區域，並更新 net 中左右的 cell 數量。

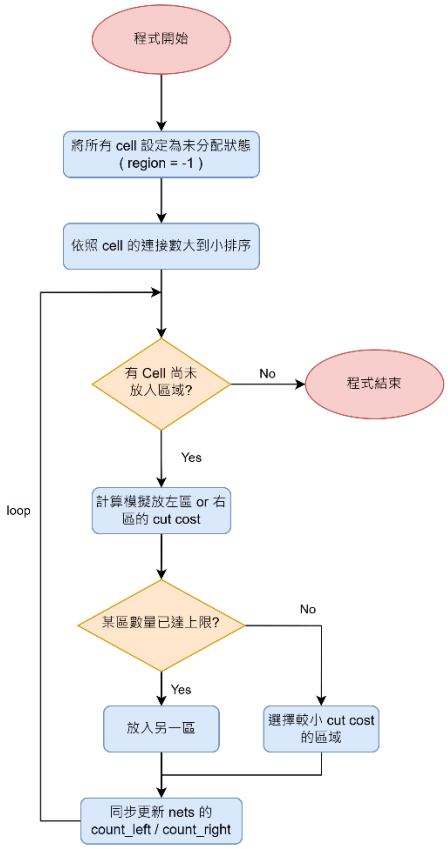
流程：

- 對每個 cell 隨機設定 region（0 或 1）。
- 同步更新：
  - count_left 和 count_right
  - 所有 cell 所屬 net 的 count_left 和 count_right

```
void Initial_Partition_Random(){
    // As initial statistics for the count of cells in each region
    count_left = 0;
    count_right = 0;

    for(auto &c: cells){
        // Randomly assign the cell's region (0 or 1)
        c.second.region = rand() % 2;
        int cid = c.first;
        // Update nets and the counts in both regions
        if(cells[cid].region == 0){
            count_left++;
            for(auto &n: cells[cid].nets)
                nets[n].count_left++;
        }
        else{
            count_right++;
            for(auto &n: cells[cid].nets)
                nets[n].count_right++;
        }
    }
}
```

b. 方法二: Initial_Partition_Greedy(): 根據 cell 的連線數量（degree）做貪婪切割初始化，優先放置高連結 cell 到 cut cost 較低的一側。



流程:

- 將所有 cell 設定為未分配狀態(region = -1)

```
for (auto &p : cells)
    p.second.region = -1;
```

- 將 cell 依照連結數由大到小排序。

```
// Sort by the degree of each cell, higher degree is prioritized
vector<pair<int, int>> cellDeg; // pair<cell_id, degree>
for (auto &p : cells) {
    cellDeg.push_back({p.first, (int)p.second.nets.size()});
}
sort(cellDeg.begin(), cellDeg.end(), cmp_value);
```

- 對每個 cell：
  - 模擬放入左或右後的 cut cost。

```
for (auto &netName : cells[cid].nets) {
    // If there are already cells assigned to the net, increase the count for that region
    int left = 0, right = 0;
    for (int neigh : nets[netName].cells) {
        if (cells[neigh].region == 0) left++;
        else if (cells[neigh].region == 1) right++;
    }
    // If no cells are assigned to the net, placing it anywhere has no cost
    if(left == 0 && right == 0) continue;
    // If placed in the left region:
    // If there are already cells in the right region, placing it in the left will increase the cost by 1
    if(right > 0)
        cost_left++;
    // Similarly for placing in the right region:
    if(left > 0)
        cost_right++;
}
```

- 根據 cost 與平衡條件決定放哪一側。

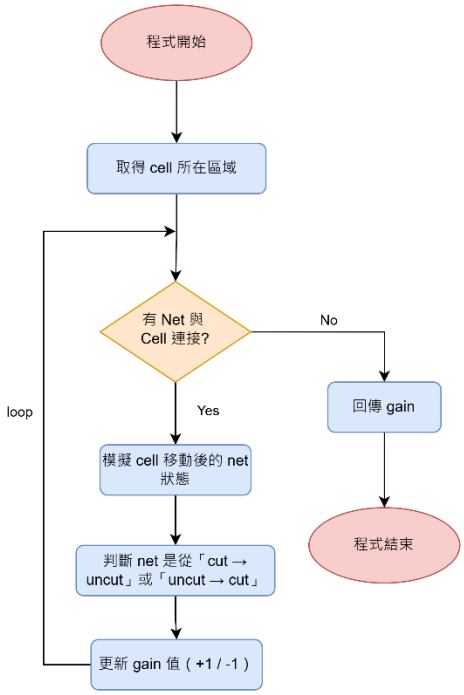
```
int chosen_region = -1;
// If the count difference in cells between regions has reached the balance limit, choose the lesser populated side
if(abs((count_left + 1) - count_right) > balance_limit) {
    chosen_region = 1;
}
else if (abs(count_left - (count_right + 1)) > balance_limit) {
    chosen_region = 0;
}
// If the cell count difference is not significant, choose the region with lower cost
else {
    chosen_region = (cost_left <= cost_right) ? 0 : 1;
}
```

- 更新：
  - cell 的 region
  - 全域 count_left / count_right
  - net 的 count_left / count_right

```
cells[cid].region = chosen_region;

// Update the count_left and count_right in nets
if(chosen_region == 0){
    count_left++;
    for(auto &n: cells[cid].nets)
        nets[n].count_left++;
}
else{
    count_right++;
    for(auto &n: cells[cid].nets)
        nets[n].count_right++;
}
```

(3) Calculate_Gain(): 計算特定 cell 的 gain 值，代表該 cell 移動後對 cut size 的正/負面影響。



流程:

- 模擬將 cell 移動到另一區。

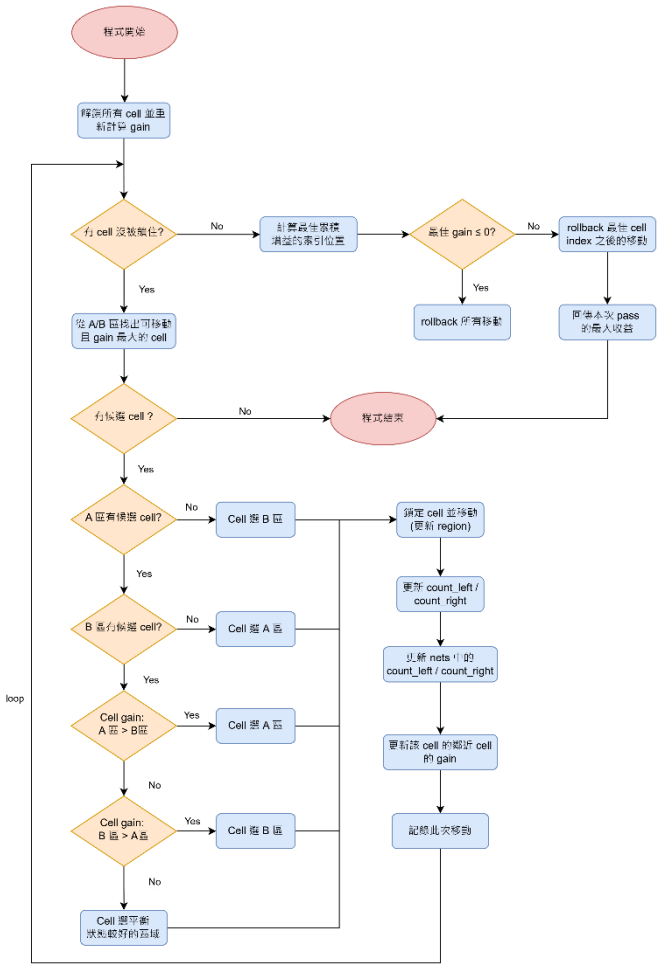
```
    // Get the current region of the cell
    int from_region = cells[cell_id].region;
    // Traverse all nets connected to the cell
    for (const auto &n : cells[cell_id].nets) {
        // Current count of cells in the net's regions
        int original_left = nets[n].count_left;
        int original_right = nets[n].count_right;
        // Simulate the count of cells in the net's regions after moving
        int tmp_left = original_left;
        int tmp_right = original_right;

        // Subtract the cell from its original region and add it to the opposite region
        if (from_region == 0) {
            tmp_left--;
            tmp_right++;
        }
        else {
            tmp_right--;
            tmp_left++;
        }
```

- 對每個 net 判斷：
  - 移動前是 cut /移動後是 uncut → gain + 1
  - 移動後是 uncut /移動後是 cut → gain - 1
- 根據變化更新 gain 值。

```
        // Check if the net is a cut net before and after the move
        bool before_cut = (original_left > 0 && original_right > 0);
        bool after_cut  = (tmp_left > 0 && tmp_right > 0);
        if (before_cut && !after_cut)
            gain++;   // From cut to uncut, gain +1
        else if (!before_cut && after_cut)
            gain--;   // From uncut to cut, gain -1
    }
    return gain;
```

(4) FM_Algorithm(): 執行單次 FM 演算法，透過移動 gain 最大的 cell，最佳化切割。



流程：

- 所有 cell 解鎖並重新計算 gain。

```
for (auto &kv : cells) {
    kv.second.locked = false;
    kv.second.gain = Calculate_Gain(kv.first);
}
```

- 重複直到所有 cell 被鎖住或無可移動者：
  - 找左/右 gain 最大且可移動的 cell。

```
int candidate_A = -1, candidate_B = -1;
int best_gain_A = INT_MIN, best_gain_B = INT_MIN;
for (const auto &c : cells) {
    int id = c.first;
    if (!cells[id].locked && cells[id].region == 0 && can_move(id)) {
        if (cells[id].gain > best_gain_A) {
            best_gain_A = cells[id].gain;
            candidate_A = id;
        }
    }
    if (!cells[id].locked && cells[id].region == 1 && can_move(id)) {
        if (cells[id].gain > best_gain_B) {
            best_gain_B = cells[id].gain;
            candidate_B = id;
        }
    }
}
```

  - 根據 gain 值和平衡策略選擇一個 cell 移動。

```
// If there are no candidates on both sides, terminate this FM pass
if (candidate_A == -1 && candidate_B == -1)
    break;
// If only one side has a candidate, choose that one
int chosen = -1;
if (candidate_A == -1)
    chosen = candidate_B;
else if (candidate_B == -1)
    chosen = candidate_A;
else {
    // If both sides have candidates, compare their gains
    if (best_gain_A > best_gain_B)
        chosen = candidate_A;
    else if (best_gain_B > best_gain_A)
        chosen = candidate_B;
    else {
        // If the gains are equal, calculate the balance after moving and
        // choose the candidate that minimizes the difference in counts between the two regions
        int new_left_A = count_left - 1;
        int new_right_A = count_right + 1;
        int new_left_B = count_left + 1;
        int new_right_B = count_right - 1;

        int diff_A = abs(new_left_A - new_right_A);
        int diff_B = abs(new_left_B - new_right_B);
        if (diff_A <= diff_B)
            chosen = candidate_A;   // If diff_A = diff_B, choose candidate_A
        else
            chosen = candidate_B;
    }
```

  - 紀錄移動操作。
  - 鎖定 cell，更新其 region。

```
// Record the gain and partition change of this move
moveSeq.push_back({ chosen, cells[chosen].gain, from, to });
// Lock the cell
cells[chosen].locked = true;
```

  - 更新所有相關 net 的 count。

```
// Update global region cell counts
if (from == 0) {
    count_left--;
    count_right++;
}
else {
    count_left++;
    count_right--;
}
// Update the count of cells in the corresponding regions for the nets the cell belongs to
for (const auto &netName : cells[chosen].nets) {
    if (from == 0) {
        nets[netName].count_left--;
        nets[netName].count_right++;
    }
    else {
        nets[netName].count_right--;
        nets[netName].count_left++;
    }
}
```

- 更新相鄰 cell 的 gain。

```
        // Update the gain values of neighboring cells sharing the same nets
        for (const auto &netName : cells[chosen].nets) {
            for (const auto &cid : nets[netName].cells) {
                if (!cells[cid].locked) {
                    cells[cid].gain = Calculate_Gain(cid);
                }
            }
        }
```

- 根據 gain 累計值找出最佳切換點。

```
    // Calculate the cumulative gain of the move sequence and find the best cumulative gain point (maximum partial sum)
    int curSum = 0;
    int bestSum = INT_MIN;
    int bestIndex = -1;
    for (int i = 0; i < moveSeq.size(); i++) {
        curSum += moveSeq[i].gain;
        if (curSum > bestSum) {
            bestSum = curSum;
            bestIndex = i;
        }
    }
```

- 如果最佳 gain ≤ 0：
  - rollback 所有移動。

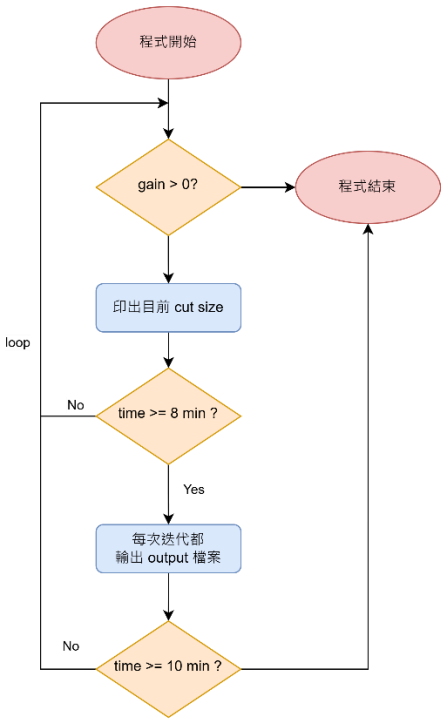
```
    // If the best cumulative gain is less than or equal to 0, rollback all moves
    if (bestSum <= 0) {
        for (int i = moveSeq.size() - 1; i >= 0; i--) {
            int cid = moveSeq[i].cell_id;
            // The original move direction was from -> to, this rollback will be in reverse
            int to = moveSeq[i].from;
            cells[cid].region = to;
            if (to == 0) {
                count_left++;
                count_right--;
            }
            else {
                count_left--;
                count_right++;
            }
            for (const auto &netName : cells[cid].nets) {
                if (to == 0) {
                    nets[netName].count_left++;
                    nets[netName].count_right--;
                }
                else {
                    nets[netName].count_left--;
                    nets[netName].count_right++;
                }
            }
        }
        return 0;
    }
```

  - 否則 rollback best gain 後的移動。

```
    else {
        // Rollback all moves from bestIndex + 1 to the end
        for (int i = moveSeq.size() - 1; i > bestIndex; i--) {
            int cid = moveSeq[i].cell_id;
            int to = moveSeq[i].from;
            cells[cid].region = to;
            if (to == 0) {
                count_left++;
                count_right--;
            }
            else {
                count_left--;
                count_right++;
            }
            for (const auto &netName : cells[cid].nets) {
                if (to == 0) {
                    nets[netName].count_left++;
                    nets[netName].count_right--;
                }
                else {
                    nets[netName].count_left--;
                    nets[netName].count_right++;
                }
            }
        }
```

- 回傳本次 pass 所得到的最大 cumulative gain

(5) Run_FM(): 重複呼叫 FM_Algorithm()，直到無法再獲得正的 gain。



流程：

- 執行 FM_Algorithm()，直到 gain ≤ 0 為止。
- 每次 pass 印出 cut size。

```
while (true) {
    int pass_gain = FM_Algorithm(cell_count);
    iteration++;
    cout << "iteration: "<< iteration << endl;
    int curr_cut_size = Compute_Cut_Size();
    cout << "Current cut size: " << curr_cut_size << endl;
    if (pass_gain <= 0)
        break;
```

- 若運行超過 8 分鐘，則立即儲存結果。
- 若運行超過 10 分鐘，結束此程式。

```
    auto TIME = chrono::high_resolution_clock::now();
    // If execution time exceeds 8 minutes, output results after each iteration
    if((float)(chrono::duration_cast<chrono::milliseconds>(TIME - START).count()/1000/60) >= 8){
        Output_Result(filename);
    }
    if((float)(chrono::duration_cast<chrono::milliseconds>(TIME - START).count()/1000/60) >= 10){
        break;
    }
}
```

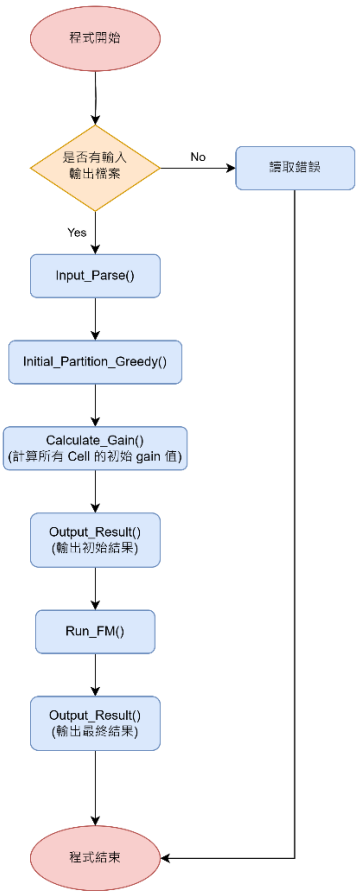
(6) Output_Result(): 將目前 partition 結果輸出到指定的輸出檔案中。

輸出格式:

```
cut_size X
A
c1
c2
...
B
c3
c4
...
```

```
// 6. Output results to the output file
void Output_Result(const string &filename) {
    ofstream outputfile(filename);
    if(!outputfile){
        cerr << "cannot open output file: " << filename << endl;
        exit(1);
    }
    int cut_size = Compute_Cut_Size();
    outputfile << "cut_size " << cut_size << "\nA\n";
    for(const auto &p : cells) {
        if(p.second.region == 0)
            outputfile << "c" << p.first << "\n";
    }
    outputfile << "B\n";
    for(const auto &p : cells) {
        if(p.second.region == 1)
            outputfile << "c" << p.first << "\n";
    }
    outputfile.close();
}
```

(7) Main function



流程:

- 檢查輸入參數
- 讀取輸入檔並解析資料
- 做 Initial Partition
- 計算初始 cut size
- 執行 FM Partitioning
- 輸出最終結果
- 顯示執行時間與統計資料

```
int main(int argc, char *argv[]){
    const auto start = chrono::high_resolution_clock::now();
    if(argc < 3){
        cout << "Usage: " << argv[0] << " input.txt output.txt" << endl;
        return 1;
    }
    int cell_count = 0;
    // Read file
    Input_Parse(argv[1], cell_count);

    // Initial Solution
    // Initial_Partition_Random();
    Initial_Partition_Greedy();
    cout << "Initial cut size: " << Compute_Cut_Size() << endl << endl;

    // Initialize the gain of each cell and output the initial result (calculate once first)
    for(auto &it : cells){
        it.second.gain = Calculate_Gain(it.first);
    }
    Output_Result(argv[2]);

    // Call Run_FM for multiple passes
    Run_FM(cell_count, argv[2], start);

    // Output final result
    Output_Result(argv[2]);

    cout << "end iteration" << endl;
    auto finish = chrono::high_resolution_clock::now();
    cout << "runtime: " << (float)chrono::duration_cast<chrono::microseconds>(finish-start).count() / 1000000  << " S" << endl;
    return 0;
}
```

(8) 資料結構

在我的程式中，主要利用兩個 struct 結構進行存儲電路資訊。

a. Cell

```
struct Cell {
    int gain = 0;
    int region = -1;    // 0: left region, 1: right region
    bool locked = false;
    unordered_set<string> nets;
};
```

每個 Cell 儲存一個 cell 的資訊，包括 gain、所屬區域、是否鎖定以及與其連接的所有 nets。

b. Net

```
struct Net {
    unordered_set<int> cells;
    int count_left = 0;     // count of cells in the left region
    int count_right = 0;    // count of cells in the right region
};
```

每個 Net 儲存一個 net 的資訊，包括與其連接的所有 cells、與其連接的 cells 分別放置左/右區域的數量。

4. The results and runtimes for the public test cases

Testcase	Initial Cut Size	Final Cut Size	Runtime (s)
case1	4	3	0.000523
case2	957	881	6.985020

- 計時方式：以 std::chrono 手動記錄起始與結束時間

5. Hardness

我覺得這次作業最困難的地方是在 FM 的整體邏輯實作與 rollback 機制上的設計。一開始我以為只要找到 gain 最大的 cell 並移動就好，但實際執行時才發現如果沒有記錄完整的移動序列與 gain 累積變化，就無法判斷何時該 rollback，也就無法保證每次迭代的最佳性。

此外我也遇到一個邏輯上的挑戰，就是在於 FM 的 gain 計算必須非常精確，要考慮搬動前後每條 net 是否從 cut 變為 uncut（gain +1），或從 uncut 變成 cut（gain -1）。這表示我不能只記住 cell 本身的區域，也要隨時更新該 cell 所屬 net 的 count_left / count_right，並根據這些資訊動態計算 gain。因此在資料結構的設計上我使用了 unordered_map<int, Cell> 來儲存所有 cell，以及 unordered_map<string, Net> 來儲存所有 net。因為 unordered_map 的查找速度較快，可以根據 cell 編號或 net 名稱快速找到對應資訊，有助於加快 FM 中大量搜尋與更新的效率，特別是在 rollback 時我需要一次把所有變更還原，若使用 array 或 vector 會需要額外搜尋，效率會變差。

6. Suggestions

這次的作業流程安排合理，說明也算清楚，尤其在 input/output 格式上有提供範例與說明圖片，讓我在設計資料結構時可以更有方向。不過我認為可以在這作業上再增加一個項目: 提供一個更明確的統計或圖形分析方向，例如「初始解 vs 最終 cut-size 的改善程度」的統計資料或圖表模板，這樣可以觀察 FM 過程能將 cut-size 從多少改善到多少，並以 bar chart 或折線圖顯示每次迭代的 gain 變化，這會讓我們更清楚 FM 的優化效果。